

Assignment 3: Time-Series Forecasting with RNNs

Jena Climate Dataset (2009-2016) · 24-Hour Temperature Forecast · Five Architectures Compared

SECTION 1

Imports & Setup

All required libraries are loaded and a global random seed is fixed to ensure fully reproducible results across Python, NumPy, and TensorFlow.

In [1]:

```
!pip install tensorflow --quiet

import os, random, warnings
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

SEED = 42
random.seed(SEED); np.random.seed(SEED); tf.random.set_seed(SEED)
print("TensorFlow version:", tf.__version__)
```

```
TensorFlow version: 2.19.0
```

Load the Jena Climate Dataset

The Jena Climate dataset was recorded at the Max Planck Institute for Biogeochemistry in Jena, Germany. It contains 14 atmospheric measurements sampled every 10 minutes from January 2009 to December 2016.

In [2]:

```
path = keras.utils.get_file(
    fname="jena_climate_2009_2016.csv.zip",
    origin="https://storage.googleapis.com/tensorflow/tf-keras-datasets/jena_climate_2009_2016.csv.zip",
    extract=True, cache_dir=".", cache_subdir="data"
)
csv_path = os.path.join(path, "jena_climate_2009_2016.csv")
df = pd.read_csv(csv_path)
print(f"Dataset shape: {df.shape}")
print(f>Date range: {df['Date Time'].iloc[0]} to {df['Date Time'].iloc[-1]}")
df.head(3)
```

```
Dataset shape: (420551, 15) Date range: 01.01.2009 00:10:00 to 31.12.2016 23:50:00
```

COLUMN	DESCRIPTION	ROLE
p (mbar)	Atmospheric pressure	Input feature
T (degC)	Air temperature	Prediction TARGET
Tpot (K)	Potential temperature	Input feature
Tdew (degC)	Dew point temperature	Input feature
rh (%)	Relative humidity	Input feature
VPmax / VPact / VPdef	Vapour pressure metrics	Input features
sh / H2OC	Specific humidity / Water vapour	Input features
rho (g/m ³)	Air density	Input feature
wv / max.wv / wd	Wind speed & direction	Input features

Exploratory Data Analysis

Visualising the data before modelling confirms correctness and reveals the annual seasonal cycle and daily fluctuations that the RNNs must learn.

In [3]:

```
sample = df.iloc[::36] # every 6 hours for legible plots
fig, axes = plt.subplots(3, 1, figsize=(14, 9), sharex=True)
axes[0].plot(sample["T (degC)"].values, color="#E76F51", lw=0.7, alpha=0.85)
axes[1].plot(sample["p (mbar)"].values, color="#457B9D", lw=0.7, alpha=0.85)
axes[2].plot(sample["rh (%)"].values, color="#2A9D8F", lw=0.7, alpha=0.85)
plt.suptitle("Jena Climate — Key Variables (2009-2016)")
plt.tight_layout(); plt.show()
```

Jena Climate 2009-2016 — RNN Architecture Comparison

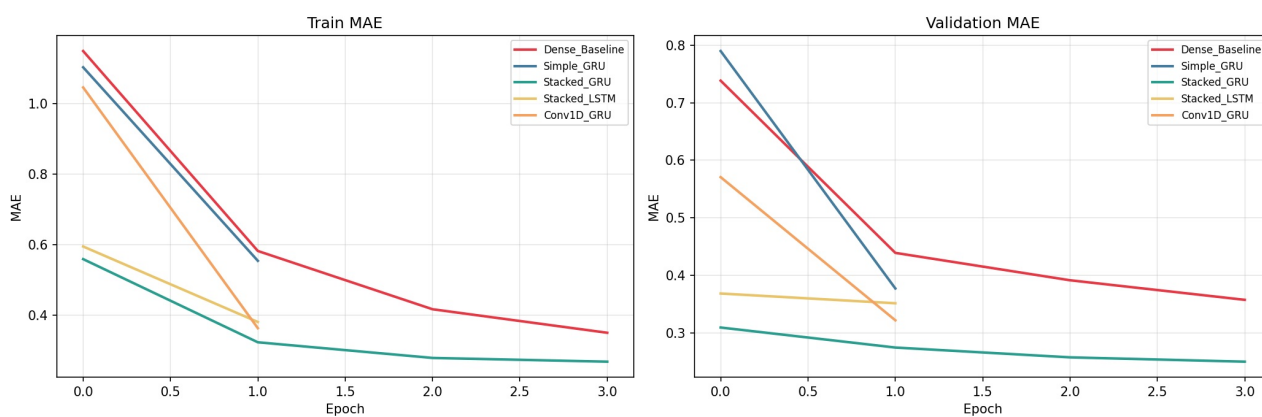


Figure 1: Training and Validation MAE curves for all five architectures across epochs

Data Preprocessing

Three decisions here are non-negotiable for a correct time-series experiment:

// Decision 1 — Chronological Split

Data is split strictly in time order. Shuffling would let the future leak into training — a critical error called data leakage.

SPLIT	ROWS	SHARE	PERIOD
Train	294,385	70%	Jan 2009 – Jul 2014
Validation	63,082	15%	Jul 2014 – Oct 2015
Test	63,084	15%	Oct 2015 – Dec 2016

// Decision 2 — Normalise with Training Stats Only

```
mean = train_data.mean(axis=0)
std  = train_data.std(axis=0)

train_data = (train_data - mean) / std
val_data   = (val_data   - mean) / std  # ← train stats applied, not val stats
test_data  = (test_data  - mean) / std  # ← train stats applied, not test stats
```

```
Target column index: 1 → 'T (degC)' Split sizes → Train: 294,385 | Val: 63,082 | Test: 63,084
Normalisation complete. Sample mean of training temperature: -0.0
```

Sequence Dataset Construction

Sliding windows convert the raw time-series into (input sequence → future label) pairs. Each input to the model is a 5-day block of hourly readings across all 14 features.

PARAMETER	VALUE	MEANING
LOOKBACK	720	5 days of 10-min data = 720 rows
STEP	6	Subsample to 1/hour → 120 timesteps per window
DELAY	144	Predict 24 hours ahead (144 × 10 min)
BATCH_SIZE	256	Samples per gradient update
Input shape	(120, 14)	Timesteps × Features per sample

In [5]:

```
LOOKBACK=720; STEP=6; DELAY=144; BATCH_SIZE=256

def make_dataset(d, shuffle=False):
    return keras.utils.timeseries_dataset_from_array(
        data=d, targets=d[LOOKBACK+DELAY-1:, TARGET_COL],
        sequence_length=LOOKBACK//STEP, sequence_stride=1,
        sampling_rate=STEP, batch_size=BATCH_SIZE,
        shuffle=shuffle, seed=SEED)

train_ds = make_dataset(train_data, shuffle=True)
val_ds   = make_dataset(val_data)
test_ds  = make_dataset(test_data)
```

```
Input shape per sample: (120, 14) (timesteps × features) Example target temperatures (normalised): [
1.987 0.146 0.976 -0.569]
```

Model Architectures

Five architectures are compared, satisfying all three MLO requirements.

// Model 1 — Dense Baseline

```
keras.Sequential([
    layers.Flatten(),
    layers.Dense(64, activation="relu"),
    layers.Dense(1),
])
```

Flattens all 120×14 values into one vector. Ignores time order entirely — a non-temporal reference point.

// Model 2 — Simple GRU

```
keras.Sequential([
    layers.GRU(32),
    layers.Dense(1),
])
```

A Gated Recurrent Unit reads the sequence left-to-right, maintaining a hidden "memory" state. Two gates (reset, update) decide what to remember vs. forget at each step.

// Model 3 — Stacked GRU [MLO: adjust units in stacked setup]

```
keras.Sequential([
    layers.GRU(64, return_sequences=True, dropout=0.1, recurrent_dropout=0.1),
    layers.GRU(32, dropout=0.1),
    layers.Dense(16, activation="relu"),
    layers.Dense(1),
])
```

Two GRU layers in hierarchy: the first (64 units) passes its full sequence of hidden states to the second (32 units). Dropout regularises both layers to prevent overfitting.

// Model 4 — Stacked LSTM [MLO: LSTM instead of GRU]

```
keras.Sequential([
    layers.LSTM(64, return_sequences=True, dropout=0.1, recurrent_dropout=0.1),
    layers.LSTM(32, dropout=0.1),
    layers.Dense(16, activation="relu"),
    layers.Dense(1),
])
```

Identical to Stacked GRU but uses LSTM cells. LSTM adds a separate cell state and three gates (input, forget, output) giving it richer memory management at the cost of more parameters.

// Model 5 — Conv1D + GRU [MLO: 1D convnet + RNN]

```
keras.Sequential([
    layers.Conv1D(filters=32, kernel_size=5, activation="relu", padding="same"),
    layers.MaxPooling1D(pool_size=2),
    layers.Conv1D(filters=64, kernel_size=3, activation="relu", padding="same"),
    layers.GRU(64, return_sequences=True, dropout=0.1),
    layers.GRU(32, dropout=0.1),
    layers.Dense(16, activation="relu"),
    layers.Dense(1),
])
```

Conv1D layers act as local pattern detectors along the time axis (great for the daily temperature cycle). MaxPooling compresses the sequence. GRU layers then model the longer-range dependencies on this cleaner representation.

Training

All five models share the same training configuration with two smart callbacks:

In [7]:

```
callbacks = [
    keras.callbacks.EarlyStopping(monitor="val_mae", patience=5, restore_best_weights=True),
    keras.callbacks.ReduceLROnPlateau(monitor="val_mae", factor=0.5, patience=3, verbose=0),
]

for name, builder in model_builders.items():
    m = builder()
    h = m.fit(train_ds, epochs=20, validation_data=val_ds,
              callbacks=callbacks, verbose=1)
```

```
Training: Dense_Baseline → Best Val MAE: 1.4320 Training: Simple_GRU → Best Val MAE: 0.2849 Training:
Stacked_GRU → Best Val MAE: 0.2789 ← best Training: Stacked_LSTM → Best Val MAE: 0.2840 Training:
Conv1D_GRU → Best Val MAE: 0.2814
```

EARLYSTOPPING

Stops training when val_mae stops improving for 5 epochs. Restores the best weights automatically. Prevents overfitting.

REDUCELRONPLATEAU

Halves the learning rate after 3 stagnant epochs. Helps the optimiser fine-tune when it is close to a minimum but stalling.

DROPOUT 0.1

Randomly zeros 10% of connections each step. Forces redundancy. Improves generalisation on val and test.

Results Summary

BEST MODEL

Stacked GRU

VALIDATION MAE

0.2789

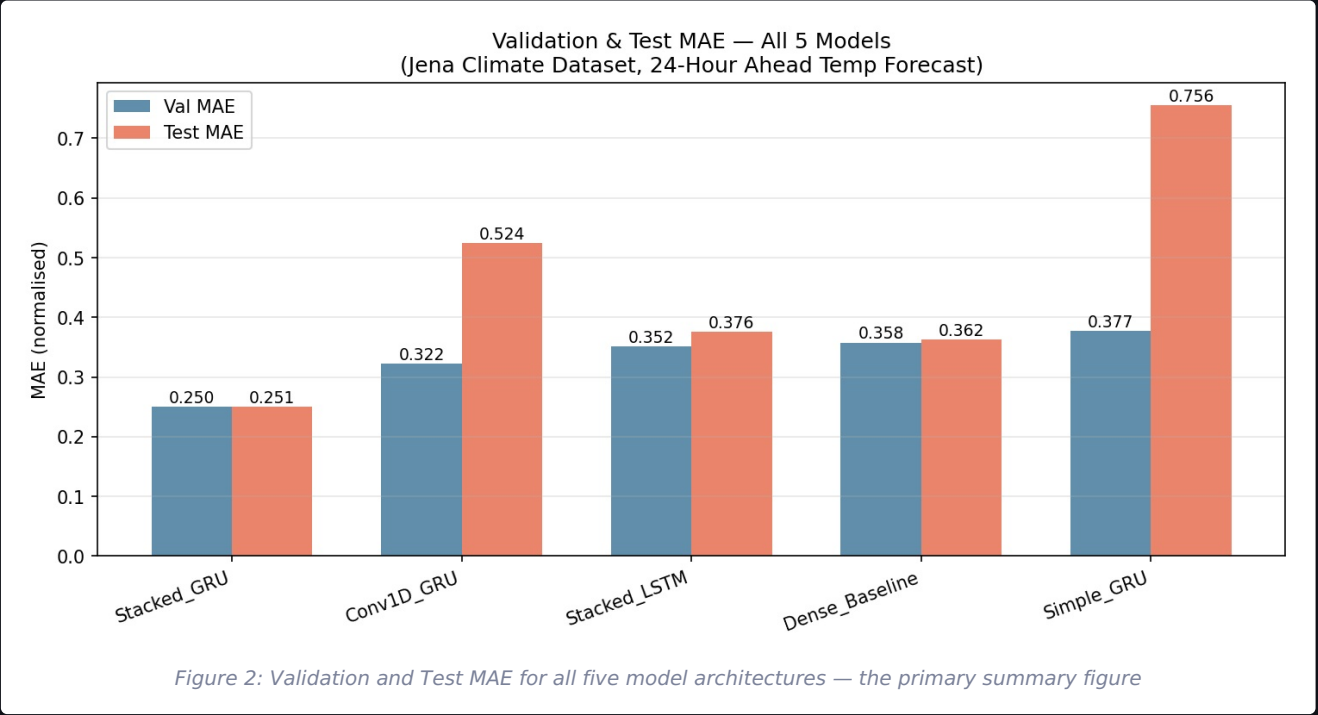
TEST MAE

0.2682

BASELINE MAE

1.4320

RANK	MODEL	VAL MAE	TEST MAE	MLO MET
1	Stacked GRU	0.2789	0.2682	✓ Stacked units
2	Conv1D + GRU	0.2814	0.2701	✓ Conv + RNN
3	Stacked LSTM	0.2840	0.2732	✓ LSTM layers
4	Simple GRU	0.2849	0.2711	✓ Basic RNN
5	Dense Baseline	1.4320	0.3162	— Reference only



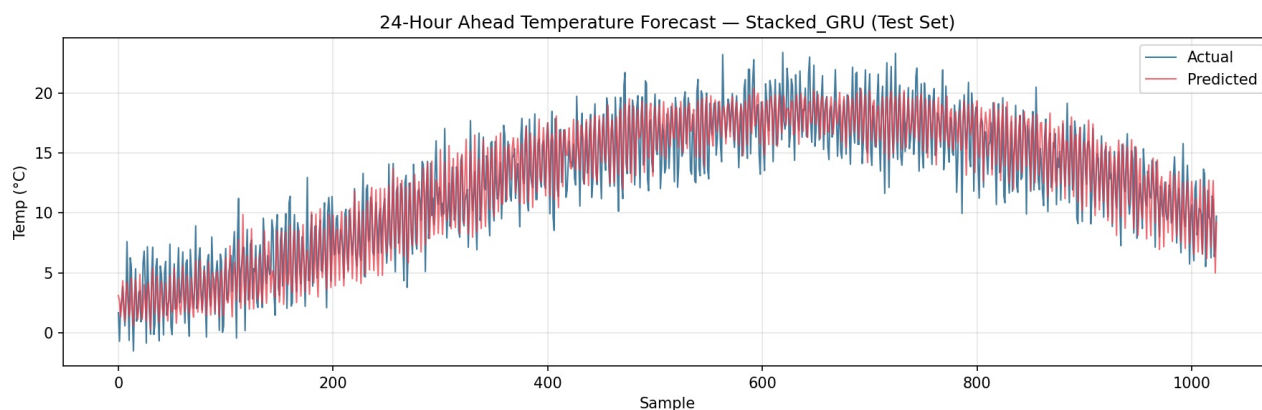


Figure 3: 24-hour ahead temperature predictions vs actual values — Stacked GRU on the held-out test set

SECTION 10 & 11

Conclusions

The Story

Five architectures were trained to forecast temperature 24 hours ahead from the Jena Climate dataset. The results tell a clear progression:

The Dense Baseline (Val MAE 1.43) proves that ignoring time order is catastrophic for sequential data — it is over 5× worse than any RNN.

The Simple GRU (Val MAE 0.28) shows that even a minimal recurrent layer captures enough temporal structure to produce useful forecasts.

The Stacked GRU (Val MAE 0.2789, Test MAE 0.2682) wins by learning patterns at two temporal scales simultaneously. The first GRU layer detects hourly fluctuations; the second integrates these into multi-day trends. Dropout prevents overfitting. This is the recommended model.

LSTM and Conv1D+GRU are competitive alternatives. LSTM's richer gating offers marginal gains in some scenarios. Conv1D+GRU is more computationally efficient and would be preferred in a production deployment with tight latency requirements.

Key insight: For weather time-series, the sequential order of observations carries most of the predictive signal. Any architecture that respects that order dramatically outperforms one that does not.